

TALLER ANALÍTICO: CALCULANDO EL GRADIENTE (20 MIN)

Suponga una matriz de imagen de 3x3 donde hay un borde vertical perfecto (mitad izquierda negra, mitad derecha blanca):

$$\begin{vmatrix} 0 & 0 & 255 \\ 0 & 0 & 255 \\ 0 & 0 & 255 \end{vmatrix}$$

1. Calcule la convolución de esta matriz con el Kernel **Sobel X (Gx)**. (Céntrese solo en el píxel del medio). ¿Cuál es el valor del gradiente en X?
2. Calcule la convolución con el Kernel **Sobel Y (Gy)**. ¿Por qué el resultado es cero? ¿Qué indica esto sobre la dirección del borde?

1. Fila 1: $(0 \times -1) + (0 \times 0) + (255 \times 1) = 255$
 Fila 2: $(0 \times -2) + (0 \times 0) + (255 \times 2) = 510$
 Fila 3: $(0 \times -1) + (0 \times 0) + (255 \times 1) = 255$
 $255 + 510 + 255 = 1020$
 El valor del gradiente en x es: 1020
2. Fila 1: $(0 \times -1) + (0 \times -2) + (255 \times -1) = -255$
 Fila 2: $(0 \times 0) + (0 \times 0) + (255 \times 0) = 0$
 Fila 3: $(0 \times 1) + (0 \times 2) + (255 \times 1) = 255$
 $-255 + 0 + 255 = 0$
 El resultado en y es: 0

Por que? Porque las tres filas de la imagen son exactamente iguales (todas dicen: 0, 0, 255). Es decir, de arriba hacia abajo no hay ningún cambio, el color se mantiene igual en cada columna sin importar la fila en la que esté.

El Kernel Gy está diseñado para detectar cambios de arriba hacia abajo, pero como aquí no hay ningún cambio en esa dirección, no encuentra nada que reportar, por eso da 0.

¿Qué nos dice esto sobre la dirección del borde?

Nos confirma que el borde de esta imagen es totalmente vertical. Como el cambio de color solo ocurre de izquierda a derecha (por eso Gx dio un número alto) y no de arriba a abajo (por eso Gy dio cero), esto demuestra que la línea que separa el negro del blanco es una línea recta y vertical

TALLER DE LABORATORIO: INSPECTOR DE BORDES (45 MIN)

Desafío Práctico:

1. Carguen una imagen que contenga formas geométricas claras y algunas texturas complejas (como un paisaje con edificios).
2. Generen tres variables separadas: Los bordes verticales usando Sobel X, los horizontales usando Sobel Y, y los bordes finales combinados usando el algoritmo de Canny.
3. Construyan un panel de visualización para comparar los tres.
4. **Experimentación:** Modifiquen los umbrales del algoritmo Canny (ej. bájenlos a 10 y 50, y luego súbanlos a 200 y 250). Observen cómo reacciona el algoritmo e identifiquen empíricamente cuáles son los umbrales óptimos para su imagen particular.

```

Laboratorio_inspector_bordes.py x 5_canny_umbral_alto.png x gato.jpg x
Taller 5 > Laboratorio_inspector_bordes.py > ...
1 import cv2
2 import numpy as np
3
4 imagen = cv2.imread('Taller 5/imagenes/gato.jpg', cv2.IMREAD_GRAYSCALE)
5
6 if imagen is None:
7     print("Error: no se pudo cargar la imagen. Verifica el nombre y la ruta")
8 else:
9     print("Imagen cargada correctamente. Forma:", imagen.shape)
10
11     sobel_x = cv2.Sobel(imagen, cv2.CV_64F, 1, 0, ksize=3)
12     sobel_x_visible = cv2.convertScaleAbs(sobel_x)
13
14     sobel_y = cv2.Sobel(imagen, cv2.CV_64F, 0, 1, ksize=3)
15     sobel_y_visible = cv2.convertScaleAbs(sobel_y)
16
17     bordes_canny = cv2.Canny(imagen, 50, 150)
18
19     cv2.imwrite('Taller 5/1_sobel_x.png', sobel_x_visible)
20     cv2.imwrite('Taller 5/2_sobel_y.png', sobel_y_visible)
21     cv2.imwrite('Taller 5/3_canny.png', bordes_canny)
22
23     print("Listo. Revisa los 3 archivos en la carpeta Taller 5:")
24     print("1_sobel_x.png -> bordes verticales")
25     print("2_sobel_y.png -> bordes horizontales")
26     print("3_canny.png -> bordes combinados")
27
28     canny_bajo = cv2.Canny(imagen, 10, 50)
29     canny_alto = cv2.Canny(imagen, 200, 250)
30
31     cv2.imwrite('Taller 5/4_canny_umbral_bajo.png', canny_bajo)
32     cv2.imwrite('Taller 5/5_canny_umbral_alto.png', canny_alto)
33
34     print("También se generaron 2 pruebas extra:")
35     print("4_canny_umbral_bajo.png -> umbrales 10 y 50")
36     print("5_canny_umbral_alto.png -> umbrales 200 y 250")

```



Laboratorio_inspector_bordes.py u x edificio.jpg u x 2_sobel_y.png u x

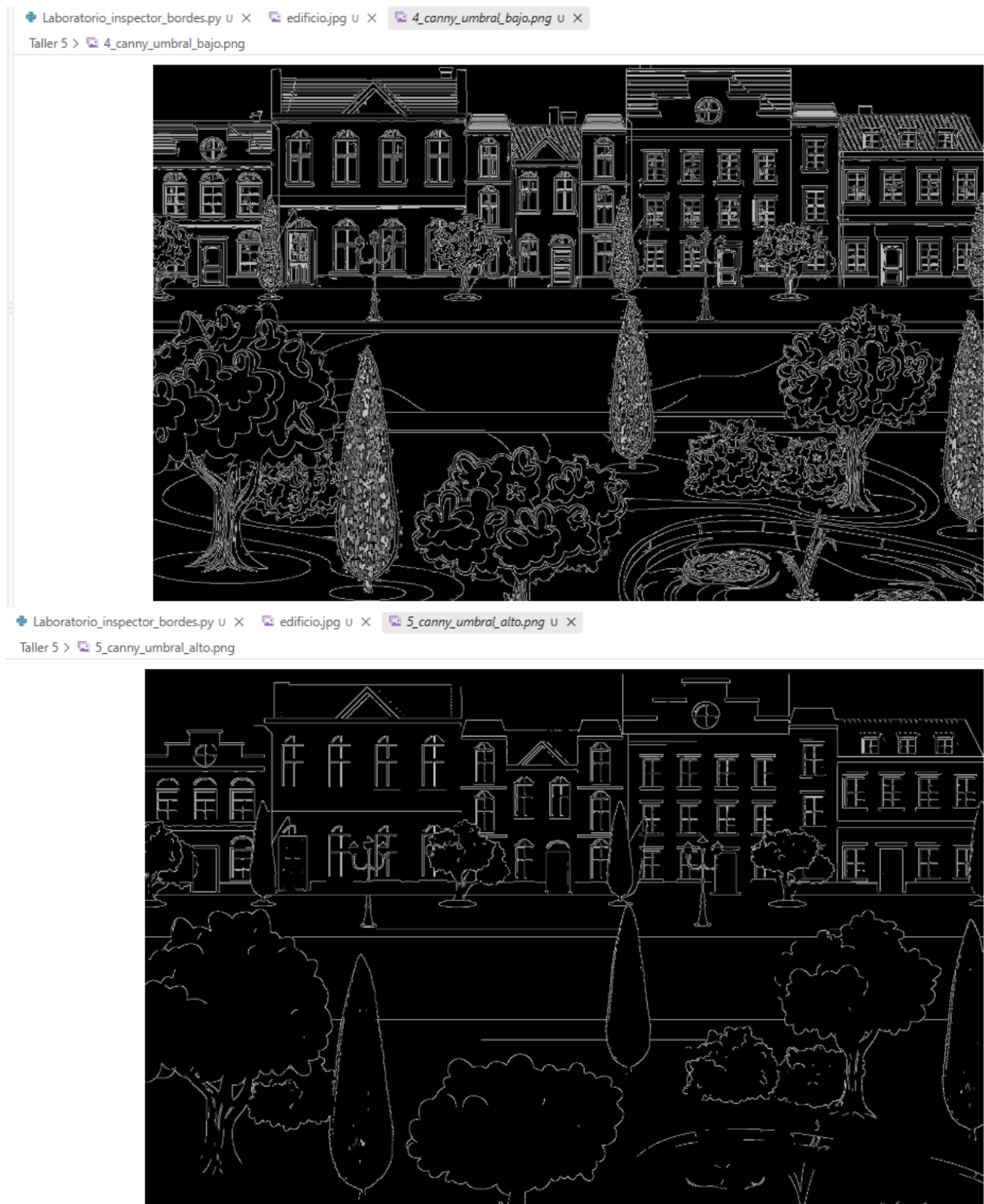
Taller 5 > 2_sobel_y.png



Laboratorio_inspector_bordes.py u x edificio.jpg u x 3_canny.png u x

Taller 5 > 3_canny.png





Análisis de los umbrales de Canny

Al bajar los umbrales de Canny a 10 y 50, el algoritmo se vuelve mucho más "sensible": empieza a marcar como bordes hasta los cambios de color más pequeños, incluyendo

texturas, sombras suaves o pequeñas imperfecciones que no son bordes reales. Esto genera una imagen con muchísimas líneas, incluso en zonas donde no había ningún objeto definido, como si el algoritmo "viera ruido donde no lo hay"

Al subir los umbrales a 200 y 250, pasa justo lo contrario: el algoritmo se vuelve muy exigente y solo marca como borde los cambios de color extremadamente fuertes. El resultado es una imagen casi vacía, donde se pierden bordes importantes que sí eran parte real de los objetos, porque el cambio de intensidad ahí no fue lo suficientemente alto para cumplir el umbral